

COURSE CODE:DCIT 208

ASSIGNMENT NUMBER: 1

ASSIGNMENT TITLE: PERSONAL SYSTEM REQUEST-FLOW DIAGRAM

NAME: HAMMOND KOTEY AMON

STUDENT ID: 22303916

TEAM NAME: LUXYBYNESS

PROJECT TITLE:MODERN SALON MANAGEMENT AND BOOKING WEBSITE

TEAM REPOSITORY URL: <https://github.com/John-owusu-ansah/DCIT-208-Group-29>

INDIVIDUAL REPOSITORY URL: <https://github.com/HammondAmon/Personal-repo>

DATE: 25TH JUNE,2026

AI DISCLOSURE STATEMENT: I used AI assistance. I used google gemini for turning my rough handwritten note into digital text. I also expanded my project outline into 1000+ expanded text.

YOUTUBE LINK: <https://youtu.be/DfKrwI6wdcg?si=dwtJ1LObCE9Q-B3y>

1. Introduction and Comprehensive System Overview

The modern retail landscape demands that service-oriented enterprises leverage highly scalable, robust, and accessible digital solutions to manage day-to-day operations. The application under design by our engineering team is a comprehensive, enterprise-grade Beauty Parlour Management System. This platform is not a singular application but an interconnected software ecosystem tailored to meet the needs of four distinct user classes: end-consumer clients, salon receptionists, professional service stylists, and business administrators.

To maximize accessibility and eliminate friction, the user will operate the platform primarily via standard web browsers across multiple hardware form factors. Clients will interact with a mobile-optimized web interface to view stylist schedules, browse the parlour's service catalog, and book appointments asynchronously on their smartphones. Simultaneously, internal salon staff—such as receptionists and business managers—will operate the platform using heavy-duty desktop hardware. This desktop environment provides access to an administrative control panel optimized for rapid data entry, real-time salon floor scheduling, point-of-sale invoicing, and backend inventory configuration. The web browser architecture ensures that no specialized local software installations are required, facilitating cross-platform compatibility across Windows, macOS, iOS, and Android devices.

2. Frontend Layout Styling and Asset Optimization

To project an aesthetic of premium luxury that matches the physical ambiance of a high-end beauty parlour, the user interface requires a sophisticated presentation layer. The team currently plans to utilize the advanced cascading stylesheet functional notation, specifically `css url()`. In a production-grade web architecture, hardcoding heavy graphical media directly into the application layout dramatically degrades initial page loading speeds and compromises user retention. By utilizing the `css url()` method, our team can separate structural layout components from heavy aesthetic assets.

This functional property will be used to dynamically pull high-definition external media files from optimized content delivery networks. These external files include high-resolution background imagery of modern salon interiors, custom branding typography font files, and dynamic promotional media banners. Furthermore, by linking external asset URLs rather than embedding heavy inline assets, the frontend application benefits from native browser caching mechanisms. Once a client loads the salon's background styling or custom fonts during their initial visit, their browser caches those files locally, guaranteeing sub-second rendering speeds for all subsequent scheduling and booking pages.

3. Microservice Computing and Serverless Cloud Scalability

On the backend infrastructure layer, the team is deviating from legacy monolithic server designs to build a modern, event-driven architecture. The AWS Lambda function URL will be used to aid in cloud computing and horizontal scalability. Traditional server architectures require a dedicated virtual machine running twenty-four hours a day, which consumes financial resources regardless of actual consumer traffic. AWS Lambda, by contrast, operates on a serverless paradigm where computing power is completely elastic and billed strictly per millisecond of code execution.

The core capabilities of the Beauty Parlour Management System are decoupled into isolated microservices. The team plans on using secure Hypertext Transfer Protocol Secure (HTTPS) endpoints for individual functions, ensuring that features like calendar scheduling, stock tracking, and employee payroll calculate inside completely isolated virtual runtimes.

The primary business justification for choosing AWS Lambda lies in its native tolerance for massive concurrent spikes in web traffic. In the beauty and wellness industry, traffic is rarely flat; it surges drastically during holiday seasons, weekend rush hours, or when marketing promotions drop. If 500 people simultaneously access the platform to book an appointment at the exact same time, a traditional server setup would experience thread starvation, bottleneck, and crash under the load. In our serverless cloud configuration, the AWS infrastructure instantly detects the concurrent traffic spike and automatically spins up 500 parallel, isolated instances of the deployment code to handle each booking request simultaneously. As soon as the traffic influx clears, the virtual instances gracefully scale down to zero, providing the parlour business with an optimal, cost-efficient computing framework.

4. The API Communication Framework and Network Transport Layer

For the comprehensive management workflow, the network transit pathway relies heavily on the Hypertext Transfer Protocol Secure (HTTPS) protocol. HTTPS is primarily used to provide a reliable, encrypted transmission medium when data leaves the frontend client browser, travels across the public internet infrastructure, and targets the backend serverless cloud layer. As HTTPS secures the network pathway via Transport Layer Security (TLS) cryptographic protocols, it prevents malicious actors from launching man-in-the-middle attacks or harvesting private consumer scheduling patterns.

While the HTTPS network layer acts as the physical highway, the Application Programming Interface (API) represents the specific vehicle moving across that road. The API structures application data into standardized JSON (JavaScript Object Notation) packages, mapping individual user clicks to explicit paths inside the backend application logic. When an action is taken—such as booking a hair coloring appointment—the frontend initiates an HTTP POST request. This request contains a precise destination address written on its packet delivery wrapper (such as `POST /api/v1/appointments/book`) and carries the user's selections cleanly encapsulated within the data payload of the request body.

5. Input Data Payloads and Request Validation Rules

When the JSON packet arrives at the AWS API gateway, it contains the critical data payload needed to process the booking request. A standard appointment JSON payload includes specific structural keys:

`userId`: A unique alphanumeric string identifying the authenticated customer.

`serviceId`: An identifier indicating the selected parlour treatment (e.g., haircut, facial, manicure).

`stylistId`: The designated staff member selected to perform the service.

`dateTime`: An ISO 8601 standardized timestamp defining the exact booking date and hour.

The component that validates this incoming request payload is the Application Logic running on the backend server. The backend code acts as an algorithmic gatekeeper, executing strict multi-stage validations before allowing any request to mutate the underlying database state. The application logic executes two vital forms of validation:

The Token Check (Authentication): The backend reads the authorization header attached to the HTTPS request to extract the cryptographic JSON Web Token (JWT). It verifies that the user is legitimately logged into the system and that their digital session signature is authentic and unexpired.

The Permission Check (Authorization): The system enforces Role-Based Access Control (RBAC). It verifies that the user role matches the administrative permissions required for the target API path. For instance, while a client has the authority to view their own bookings, they are entirely blocked from executing administrative routes like updating payroll or altering global parlour price structures.

In case validation fails—whether due to an expired login session, missing fields, or an unauthorized privilege escalation attempt—the backend architecture immediately views the attempt as an operational system threat. The backend code guards the system by instantly halting the request execution chain and short-circuiting the entire transactional workflow. The request is rejected immediately at the application logic layer, entirely preventing it from touching the database. The server then jumps straight to generating a failure response package, returning a specific HTTP error status code (such as 400 Bad Request or 403 Forbidden) with a detailed error message describing the validation failure.

6. Relational Database Design and Structured Data Tables

The management system the team desires to build will definitely touch the core database since operational data like appointments, service configurations, employee shifts, and billing invoices will be constantly added, modified, and checked out as complete. To manage these highly structured business records, the team plans on using a Relational Database Management System (RDBMS), such as PostgreSQL or MySQL. Relational databases are chosen over NoSQL

options for this enterprise system because they enforce strict transactional data integrity and prevent data anomalies.

Within a relational database model, data is organized into explicit, tabular structures where individual tables are strictly linked together using primary and foreign key constraints. [13:44, 26/06/2026] Hammond: When a client checks out at the front desk, the application logic executes a relational transaction. In a single execution block, the system updates the status row in the Appointments table to "complete", calculates any stylist commission rates, creates a corresponding record in the Invoices table, and automatically decrements the stock_qty in the Inventory table if retail shampoo or styling products were purchased during checkout.

7. Fault Tolerance, Error Packaging, and UI Presentation

In real-world cloud environments, individual infrastructure components will inevitably face unexpected failures. In case the core database goes down or suffers from temporary network latency, the application logic is engineered to implement strict fault tolerance protocols. Without defensive coding practices, an unresponsive database will cause the backend backend processing threads to hang indefinitely, consuming system memory and triggering a total crash of the hosting infrastructure.

To prevent this architectural failure, all database queries inside our AWS Lambda application logic are wrapped inside structured error-handling exceptions, known as try-catch blocks. In case the database connection times out or fails completely, the backend application logic catches the connection exception gracefully, halts further downstream processing, and immediately short-circuits to the response generation layer.

The server packages a response via HTTP back through the network highway to the user's phone or desktop with the help of a generic JSON data payload. Instead of exposing raw database error dumps—which could give hackers insights into internal database ports or server architecture—the API wraps the failure inside an HTTP 500 Internal Server Error status code containing a clean, non-revealing payload:

[13:44, 26/06/2026] Hammond: {

"success": false,

"error": "INTERNAL_SERVER_ERROR",

"message": "We are experiencing technical difficulties. Please try again later."

}

Once this HTTP response code travels down the network transport highway, the frontend application parses the payload. The client-side code stops the loading icons and renders a localized error message to the user: "Oops! Something went wrong on our end. Please try

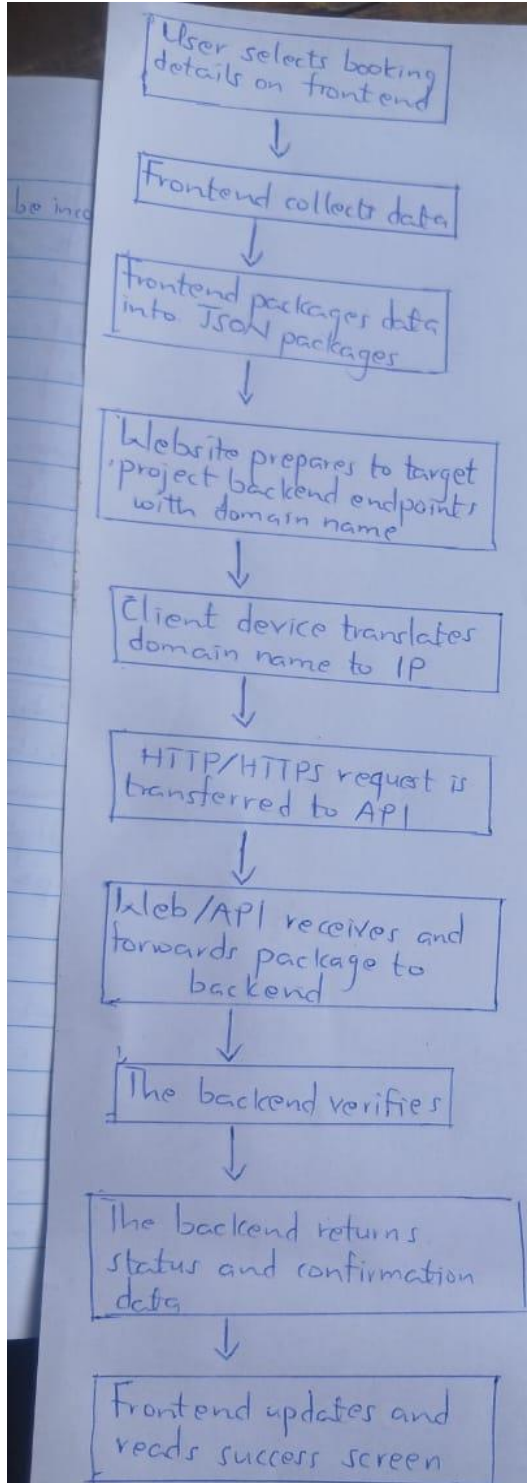
again in a few minutes." This keeps the user interface interactive and responsive, protecting the business from looking broken even during backend infrastructure downtime.

8. Financial Integration and Cyber Security Mitigation

One of the most critical security concerns the development team is deeply aware of is the persistent threat of malicious entities attempting to breach or hack the payment processing system. Because financial transaction portals handle valuable payment data, they represent a high-stakes target for automated web scraping and cyberattacks. Therefore, strict architectural security measures will be implemented if that particular payment feature is deployed by the team.

To eliminate the business risk of holding high-value payment card data, the system architecture will isolate financial processing entirely away from our core relational database tables. The team intends to build an integration with a PCI-DSS compliant third-party payment gateway API provider, such as Stripe or PayPal.

Under this design, when a receptionist processes a payment, the customer's payment details are transmitted directly from the client interface to the payment gateway's secure servers over an isolated HTTPS connection. The external gateway processes the transaction and returns a single-use, cryptographically signed token back to our AWS Lambda backend application logic. Our database only records this transaction token alongside the billing invoice record, completely ensuring that no raw credit card details or bank numbers are ever stored inside our local databases. This system design limits our threat vector, ensuring the beauty parlour management platform remains safe, compliant, and thoroughly protected against data breaches.



Name: Hammond Kotey Amon
ID: 22303916

Team name: LUXYBYNESS

Project title: Modern Salon Management and Booking Website

Date: 25th June, 2026

Time: 11:44am

Chosen feature: booking an appointment

One Possible Failure

The system might face some problems if multiple users click the "book appointment" at the exact time for the same time slot. The system might avail the slot to the multiple users. Hence, it may bring about confusion.

One Security Concern

Hackers might try to inject a malicious code into the system's database. This would allow the hacker to gain access to booking data and some secret notes from the user.